Neural Architecture Search for Edge Devices via Cloud-Edge Collaboration

Te-Yi Kan [1] **Konstantinos Psounis** [1]

Abstract

Deep neural networks (DNNs) deliver strong performance in vision and NLP, but their compute and memory demands often exceed edge devices' capabilities. Offloading to remote servers can alleviate on-device cost, but it requires transmitting user data, raising privacy concerns, and coupling performance to connectivity. Hardware-aware neural architecture search (HW-NAS), on the other hand, offers a principled way to optimize on-device model utility under latency and memory constraints. However, existing HW-NAS faces a tradeoff: cloud-centric pipelines depend on surrogate cost models and public datasets, which can mispredict on-device latency/memory and adapt poorly to runtime variability, while fully on-device search is impractical due to limited computing resources. To address these limitations, we propose a two-stage cloud–edge collaborative HW-NAS framework that trains a weight-sharing supernet and extracts a compact subnet in the cloud, then refines it on-device using private data and real hardware profiling to produce an edgenet that satisfies current constraints. Experiments show that our framework reduces search time by $3.3\times$ on average with a small performance hit versus impractical baselines on image classification and language modeling.

1. Introduction

Driven by recent advances in deep learning, DNNs have achieved strong performance across vision and NLP tasks (Zhao et al., 2019; Da'u & Salim, 2020), accelerating their adoption in real-world applications. However, many high-utility DNNs are designed for resource-rich servers and are therefore deployed via server-centric inference, which requires transmitting user data off-device, raises privacy concerns, and depends on connection availability. For delay-sensitive applications (e.g., real-time perception), the added network latency and jitter can make offloading infeasible, even when connectivity is available. Privacy-preserving techniques such as homomorphic encryption (Acar et al., 2018) can mitigate leakage, but often incur substantial overhead and may still reveal information. These limitations motivate efficient edge-oriented models that run reliably on resource-constrained devices while satisfying hardware and application latency constraints.

Prior work improves DNN efficiency through handcrafted compact models (Howard et al., 2017; Sandler et al., 2019; Howard et al., 2019) and model compression (Deng et al., 2020; Cheng et al., 2020). While these approaches can yield strong utility–efficiency tradeoffs, handcrafted designs require substantial expert effort and are often tuned to specific device characteristics, limiting their portability across heterogeneous hardware, whereas compression typically relies on proxy cost metrics (e.g., FLOPs) that can misalign with actual latency and memory on target devices. Hardware-aware neural architecture search (HW-NAS) instead automatically discovers models that maximize utility under deployment constraints such as latency and memory (Benmeziane et al., 2021b; Chitty-Venkata & Somani, 2022). However, most HW-NAS pipelines are cloud-centric and depend on surrogate cost models (e.g., latency/memory predictors) and public datasets, which can lead to deployment gaps under hardware variability and local data shifts. Moreover, they often assume static operating conditions; under time-varying edge resources, selected models may become suboptimal or even violate latency/memory budgets at runtime (Yang et al., 2020; Wen et al., 2023). Thus, fast adaptation under time-varying device conditions is essential. However, cloud-based adaptation can incur unpredictable network delays, and surrogate errors can trigger failed deployments and repeated re-search. In contrast, fully on-device HW-NAS uses real profiling and local data but is often infeasible due to limited compute on edge devices.

To address these challenges, we propose a two-stage cloud–edge collaborative HW-NAS framework that adapts to each device's local data and time-varying constraints. We construct and train a weight-sharing *supernet* from a task-specific base model, spanning diverse architectural dimensions (e.g., operator choice and width). We then perform a high-capacity architecture search using public data and an

[*]Equal contribution [1]Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA 90089 USA. Correspondence to: Te-Yi Kan <teyikan@usc.edu>.

Preprint. October 5, 2026.

estimated hardware-cost model to extract a compact *subnet* that retains only the most promising design choices, thereby substantially shrinking the downstream search space. In the edge stage, we refine this cloud-proposed subnet directly on the target device using private on-device data and real hardware profiling, producing a final *edgenet* that satisfies the device's current latency and memory constraints. Importantly, the edge-stage refinement is lightweight and can be re-run as device workloads change, enabling adaptation to time-varying conditions. As a result, our framework keeps sensitive data on-device while selecting models that remain aligned with the device's current operating condition.

We evaluate our framework on vision and NLP tasks, searching over CNN-based and Transformer-based architectures to demonstrate broad generalizability across tasks and model types. Specifically, we consider two foundational tasks, image classification and language modeling, which serve as building blocks for downstream applications such as object detection and machine translation. Our results show that the proposed framework reduces search time between $2.7\times$ and $3.8\times$ compared to baselines, while incurring at most a 5.03% drop in image classification accuracy and at most a 14.72% increase in language modeling perplexity. Importantly, these baselines are impractical in our target setting because their high search costs prevent them from adapting fast enough under time-varying device conditions given application requirements (see Section 5). In addition, they either rely on public data or require uploading local data for cloud-side evaluation; When public data are sizably misaligned from local data and/or they are not available due to privacy concerns (Zhang et al., 2025; Sriramalakshmi et al., 2022), e.g., medical data (Khalid et al., 2023; Williamson & Prybutok, 2024), they may perform poorly. Last, heavily optimized public models, which are small enough to fit on a device, such as MobileNet, are not a viable alternative in this setting, as they typically rely on large-scale pretraining, extensive tuning, and distillation, which do not transfer reliably across heterogeneous devices and shifting local data.

To summarize, the contributions of this paper are as follows:

- We study a challenging edge deployment setting in which the cloud cannot access private on-device data, and edge resources fluctuate over time due to dynamic operating conditions.
- We propose a two-stage cloud–edge collaborative HW-NAS framework that identifies competitive models tailored to the device's current operating conditions and local data distribution.
- We design an efficient on-device search pipeline, incorporating caching, stage-wise evaluation, and intermediate feature reuse, enabling efficient re-optimization as conditions change.
- We evaluate on vision and NLP tasks (image classification and language modeling) across CNN and Transformer families, demonstrating strong performance and broad applicability.

2. Problem Formulation

We formulate the design problem as a constrained architecture search, aiming to discover models that achieve high utility while satisfying latency and memory constraints on edge devices. Note that, in NAS, a candidate model design is typically referred to as an "architecture"; accordingly, we use "architecture" to denote a candidate model throughout the paper.

Formally, given an architecture a, let $U(a)$ denote its task utility, $M(a)$ denote memory usage, and $L^{\mathrm{j}}(a)$ denote type-j latency on the target edge device, where $\mathrm{j} \in \mathrm{J}$ and J is the set of latency metrics of interest during inference (defined below). Our objective is:

$$\max_{a} \quad U(a) \tag{1a}$$

$$\text{s.t.} \quad L^{\mathrm{j}}(a) \le L^{\mathrm{j,max}}, \forall \mathrm{j} \in \mathrm{J} \tag{1b}$$

$$M(a) \le M^{\max} \tag{1c}$$

Note that the utility function $U(\cdot)$ is task-specific: for image classification, $U(\cdot)$ can be accuracy, whereas, for language modeling, it can be perplexity. The latency metric $L^{\mathrm{j}}(\cdot)$ is also task-dependent: for non-autoregressive tasks, $L^{\mathrm{j}}(\cdot)$ can be a single end-to-end inference latency $L^{\inf}(\cdot)$ (Yang et al., 2018), while for autoregressive NLP tasks, latency is better characterized by both time-to-first-token (TTFT), $L^{\mathrm{TTFT}}(\cdot)$, and time-per-output-token (TPOT), $L^{\mathrm{TPOT}}(\cdot)$ (Hong et al., 2025).

As noted in Section 1, we study Problem 1 in a challenging edge setting where (i) the cloud cannot access private on-device data or profile the target device, so cloud-side search must evaluate utility on public data and estimate latency/memory via surrogates; and (ii) device resources vary over time due to system conditions, so an architecture that is feasible at one moment may later become suboptimal or even violate constraints. This motivates on-demand adaptation: when conditions shift, the device should quickly re-select an architecture that matches the current state.

3. Methodology

In this section, we describe the key components of our proposed two-stage cloud–edge collaborative HW-NAS mechanism. Figure 1 illustrates the overall framework, which consists of two stages: 1) the cloud stage (Section 3.1) and 2) the edge stage (Section 3.2). At a high level, the mechanism operates as follows: In the cloud stage, we first construct a *supernet* from a base network using the supernet construc-

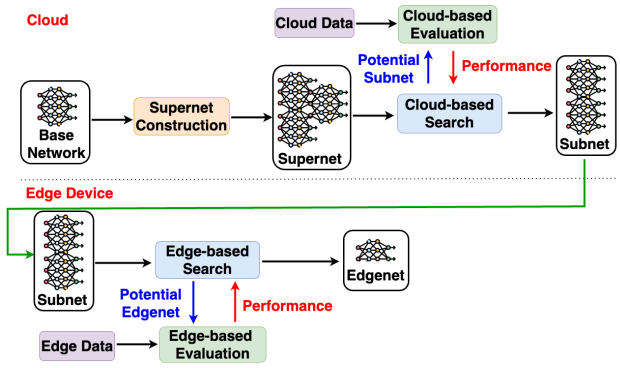


Figure 1. The overall framework of our proposed two-stage cloud-edge collaborative HW-NAS mechanism.

tion module. We then perform cloud-based evaluation and search using only cloud-available data to identify a reduced set of promising architectures (denoted as *subnet*) that is expected to achieve high utility under estimated hardware costs. The resulting subnet is then transferred to the target edge device. In the edge stage, we perform lightweight on-device search and evaluation within the subnet using local edge data and direct profiling, refining it into a model (denoted as *edgenet*) that meets the device's current workload and actual latency/memory constraints.

3.1. Cloud Stage

The cloud stage starts from a base network and outputs a compact subnet for on-device refinement. It consists of: (i) weight-sharing supernet construction (Section 3.1.1), and (ii) cloud-side search using only cloud-available data and surrogate models (Section 3.1.2), since the cloud cannot access local data or profile on the target device. By training a single supernet and extracting a reduced subnet, the cloud stage shrinks the search space before edge refinement.

3.1.1. Supernet Construction

Inspired by the once-for-all paradigm (Cai et al., 2020), given a base network n, we construct a weight-sharing supernet $\mathcal{S}(n)$ that induces an architecture space $\mathcal{A}$. This supernet paradigm allows us to train once and later evaluate many architectures efficiently by activating only the selected paths, and it naturally supports structured subnet extraction in our two-stage pipeline by pruning underperforming operators/branches from the supernet to form a compact subnet. Specifically, we replace each mutable block of network n with a super-block that contains multiple candidate choices. Formally, $\mathcal{A}$ comprises multiple dimensions, including operator choice (e.g., MBConv vs. Identity in CNNs, or a standard Transformer block vs. an attention-only variant), depth of the block, and width-related settings (e.g., channel scaling, the number of attention heads, and FFN expansion ratios). Any architecture $a \in \mathcal{A}$ is then obtained by selecting, at each super-block, a specific candidate operator together with its associated parameters (e.g., an MBConv block with a 7×7 kernel and a $1.25\times$ width multiplier), while reusing the shared parameters of $\mathcal{S}(n)$. Note that, to ensure composability across different choices, we enforce all architectures share the same input/output feature dimensions within a super-block, so that different architectures can be switched on-the-fly during training.

After building the supernet, we train $\mathcal{S}(n)$ using a one-shot procedure that samples and updates multiple architectures per mini-batch using the sandwich rule (largest, smallest, and random architectures) (Yu & Huang, 2019),. This stabilizes optimization by anchoring training at the extremes while improving coverage of intermediate architectures, yielding shared weights that transfer better across architectures. To mitigate sampling bias, we further apply fair operation sampling by upweighting under-trained architectures. Furthermore, we adopt a progressive shrinking schedule (Cai et al., 2020), where smaller architectures are parameter slices of the complete architecture (e.g., fewer layers, narrower width, fewer heads, smaller FFN); we train larger configurations first and progressively include smaller ones so they inherit well-trained representations rather than competing from scratch.

After supernet construction, we obtain a trained weight-sharing supernet $\mathcal{S}(n)$. In the next step, cloud-based search uses the shared weights to evaluate and rank architectures without retraining them from scratch.

3.1.2. Cloud-based Search

With a weight-sharing supernet, supernet training and architecture search are decoupled, making the search stage modular: we can swap in different search algorithms (e.g., evolutionary search, RL, or Bayesian optimization) without changing the training procedure, and incorporate new methods as needed. In this work, we use evolutionary search because it is robust to discrete operator choices and performs well in HW-NAS (Cai et al., 2020; Sinha et al., 2023).

The evolutionary search (Liu et al., 2023) iterates over generations by mutating architectures in $\mathcal{A}$, evaluating each architecture with shared weights for utility and surrogate models for hardware cost, and retaining feasible top-performing architectures to form the next population. This loop efficiently explores the discrete, high-dimensional search space while maintaining diversity through stochastic variation operators. Importantly, the cloud stage cannot access the edge data or profile architectures on the target device. Therefore, utility is evaluated on cloud-available data, and latency/memory are estimated using a lookup-table surrogate built offline by benchmarking representative operators/blocks on a reference edge device and aggregating per-operator costs along the selected path to approximate architecture-level costs.

Formally, during search, architectures are ranked primarily by their utility subject to hardware feasibility (i.e., we filter out architectures that violate the hardware constraints). However, to retain more potentially good architectures and to mitigate uncertainty in surrogate cost estimates, we relax the constraints by a small ratio $\rho > 0$. Specifically, $L^{\mathrm{j,max}}$ and $M^{\max}$ in constraints (1b) and (1c) are replaced by $(1+\rho)L^{\mathrm{j,max}}$ and $(1+\rho)M^{\max}$, respectively. After the evolutionary search terminates, we form the subnet by retaining, for each super-block, the operator choices appearing in the top-N architectures. The parameter N controls the subnet granularity: a larger N preserves more promising architectures but increases cloud-to-edge transfer, on-device storage, and edge-search cost, while a smaller N reduces overhead at the risk of discarding competitive architectures. The resulting subnet is thus a reduced weight-sharing search space that prunes consistently weak branches while preserving a compact set of strong, hardware-feasible architectures for on-device refinement.

3.2. Edge Stage

After receiving the subnet, the edge stage performs an on-device refinement to select a single final edgenet that satisfies the true latency and memory constraints; i.e., it aims to solve Problem (1) without any relaxation. This stage is designed to be restartable: when device workload or operating conditions change (e.g., background load), we rerun only the edge stage to quickly re-select a well-matched architecture without repeating cloud training or search. Unlike the cloud stage, the edge has access to local data and can execute architectures on the target device, enabling accurate utility evaluation and direct profiling of latency/memory.

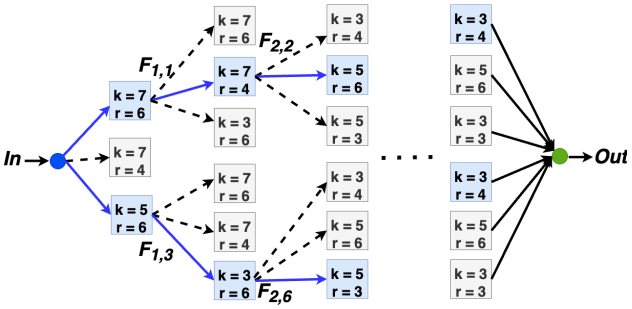


Figure 2. Sequential beam search (beam width $b = 2$; each block has 3 candidate options). The search grows the network one block at a time while maintaining the top-b partial architectures. Each rectangle represents a candidate block choice parameterized by kernel size k and expansion ratio r. Blue boxes and solid blue arrows indicate candidates retained in the beam, whereas gray boxes and dashed black arrows denote discarded candidates. For evaluation, each retained partial architecture is completed to the output node (solid black connections) to obtain its utility.

Due to the limited compute and memory budgets on edge devices, the edge-stage search must be lightweight so it can be quickly re-run as operating conditions change. Existing HW-NAS algorithms (e.g., (Fayyazi et al., 2025; Sukthanker et al., 2025)), as discussed in Section 6, are primarily designed for resource-rich platforms and often incur substantial training and evaluation cost, making them impractical for on-device refinement. To enable efficient on-device search, we build on the beam-search paradigm (Russell & Norvig, 2021) and adapt it to our setting. In traditional NAS beam search, the algorithm starts from an initial architecture with fixed design choices (e.g., kernel size and expansion ratio). At each iteration, it randomly selects a small portion of the architecture (such as one or multiple layers/blocks), expands each architecture in the current beam into multiple candidates by enumerating all available options for that portion, and retains the top-b candidates according to the search score, where b is the beam width. The expansion size depends on the number of option combinations. For example, selecting a layer with 3 kernel sizes and 2 expansion ratios produces 6 candidates in that iteration. However, vanilla beam search can still be expensive on edge devices because evaluating each candidate typically requires a full forward pass, which is too costly for rapid on-device refinement.

To further reduce on-device overhead, we modify beam search with a greedy construction procedure and call the resulting method sequential beam search (Figure 2). Instead of repeatedly editing a complete architecture, sequential beam search grows architectures block-by-block while maintaining a beam of the top-b partial architectures. It starts from an empty architecture, enumerates candidate options for the first block, and keeps the top-b candidates as the initial beam. It then expands each beam element by adding options for the next block, again retaining the top-b candidates, and repeats this process until reaching the maximum depth or until the resulting architectures consistently exceed the latency or memory budgets. As illustrated in Figure 2, at each step, the search enumerates all candidate block configurations but retains only the top-2 partial architectures (blue boxes connected by solid blue arrows). These surviving prefixes are then expanded to the next block.

The key advantage of sequential beam search is its prefix-preserving updates: candidates expanded from the same parent share a long common prefix, enabling reuse of intermediate activations from previous steps (e.g., $F_{1,1}$ and $F_{1,3}$ in Figure 2). As a result, evaluation can recompute only the newly added block rather than running a full end-to-end forward pass for every candidate, substantially reducing per-candidate cost (See the discussion below for more details). Moreover, restricting expansions to a single block limits the branching factor and reduces the total number of evaluations. The main limitation is reduced exploration compared to the vanilla beam search, since the search becomes greedy and sequential; however, this tradeoff is well aligned with our goal of fast, restartable refinement.

As noted earlier, a central challenge in on-device HW-NAS is that evaluation cost dominates: evaluating each architecture from scratch can be prohibitively expensive. We therefore use an efficient evaluation pipeline that consists of three primary mechanisms:

Caching-based acceleration. During edge search, we cache each evaluated architecture with its measured utility and hardware metrics. Because on-device performance can vary significantly with system conditions (e.g., workload and memory pressure), we store context information alongside each measurement. Upon restart, we reuse results from architectures evaluated in similar contexts, if available, to avoid repeated on-device measurements.

Stage-aware evaluation. If no suitable cached entry is available, either because the architecture has not been evaluated before or because prior measurements were collected under a mismatched system context, we evaluate the architecture via on-device inference using a stage-aware schedule: most architectures are filtered with low-fidelity estimates, and only promising ones receive higher-fidelity evaluation as the search narrows. Specifically, we use a few mini-batches in early iterations for coarse ranking, and progressively increase the number of batches to obtain more accurate utility estimates for the remaining architectures. This reduces average evaluation time while preserving reliable rankings among top architectures.

Intermediate feature reuse. Since sequential beam search generates child architectures that differ only in a short suffix, we reuse intermediate activations from the shared prefix and recompute only the modified suffix. We estimate latency/memory by combining cached prefix runtime/memory with measured suffix runtime/memory, respectively. For utility, we feed cached activations into the modified suffix to obtain updated outputs, avoiding a complete forward pass from the input. This prefix-based feature reuse substantially reduces repeated computation, especially for deep networks, and motivates our use of sequential, prefix-preserving beam search rather than non-sequential vanilla beam search.

4. Evaluation

4.1. Evaluation Setup

We implement our cloud–edge HW-NAS and all baselines in PyTorch 2.7.0 with CUDA 12.8. Experiments are conducted on a server with 2× NVIDIA RTX Pro 6000 Blackwell GPUs as the cloud platform, and on an Apple MacBook Pro with an M3 chip as the edge device. We consider two tasks: image classification and language modeling. In the cloud stage, we run evolutionary search with a population of 50 and tournament size of 5, using a mutation rate of 0.15, crossover rate of 0.7, and an elite ratio of 0.15, which fall within commonly used hyperparameter ranges in prior work (Ying et al., 2019; Lu et al., 2019b; Cai et al., 2024). We then extract a compact subnet by retaining the top-50 architectures. On-device, we perform refinement using sequential beam search with $b = 1$ and enable caching, stage-aware evaluation, and intermediate feature reuse. Note that we also study the impact of cloud–edge search configurations in an ablation study (See Section 4.5).

4.2. Baselines

We compare our cloud–edge HW-NAS against three HW-NAS baselines that run entirely in the cloud and output a final architecture without on-device refinement:

Evolutionary HW-NAS (Evo-HW-NAS): This baseline uses a standard evolutionary search with tournament selection, mutation, and crossover, and returns the best feasible architecture (highest utility under the hardware constraints).

RL HW-NAS (RL-HW-NAS): This baseline uses a PPO-based policy to sample architecture decisions and returns the best feasible architecture. Each sampled architecture is scored by the cloud evaluator with a scalar reward that reflects utility and penalizes constraint violations.

Bayesian Optimization HW-NAS (BO-HW-NAS): This baseline encodes each architecture as a fixed-length one-hot vector, fits a Gaussian-process surrogate to predict utility, and iteratively selects architectures by maximizing a UCB acquisition function; it returns the best feasible architecture under the hardware constraints.

Note that all cloud-only baselines evaluate utility using only cloud-available data and share the same lookup-table-based hardware surrogate for latency and memory. This lookup-table-based cost model is a standard and widely adopted surrogate in HW-NAS pipelines (Chitty-Venkata & Somani, 2022; Cai et al., 2019; 2020).

4.3. Image Classification

Table 1. Vision supernet construction configuration.

Search Space	Options
Base network	`MobileNetV2`
depth multipliers	$\{0.75,\ 1.0,\ 1.25\}$
width multipliers	$\{0.75,\ 1.0,\ 1.25\}$
kernel sizes (k)	$\{3,\ 5,\ 7\}$
expansion ratios (r)	$\{3,\ 4,\ 6\}$
Squeeze-and-Excitation Ratio	$\{0,\ 4\}$

In image classification, we build a MobileNet-style supernet since the MobileNet family is widely adopted for its strong accuracy–efficiency tradeoff on edge platforms (Howard et al., 2017). Following common supernet design practices (Cai et al., 2020; Yuan et al., 2024), we introduce elasticity over depth, width, and operator parameters, as summarized in Table 1. We use classification accuracy (Acc.) as the utility metric and constrain the edgenet with an inference-latency budget of 15 ms and a memory budget of 25 MB. These budgets reflect realistic edge deployment targets for image classification, where latency in the teens of millisec-

onds and memory in the tens of MB are commonly considered (Cai et al., 2020; Li et al., 2024). The cloud and edge datasets are CIFAR-100 and CIFAR-10, respectively, following their standard use in image classification benchmarking (Xue et al., 2025; Yin et al., 2023; Cai et al., 2024).

Table 2. HW-NAS performance on image classification

Framework	Acc. (%)	ΔAcc. (%)	Runtime (min)	ΔTime (%)
Our framework	87.5	-	10.45	-
Evo-HW-NAS	91.9	↑ 5.03	39.64	↑ 279.3
RL-HW-NAS	90.2	↑ 3.09	35.09	↑ 235.8
BO-HW-NAS	89.4	↑ 2.17	28.45	↑ 172.2

Note: We report accuracy on CIFAR-10 (edge data) as the utility metric for image classification and runtime as the HW-NAS search time. ΔAcc. and ΔTime denote the relative change with respect to our framework.

Table 2 shows that our framework achieves the lowest HW-NAS search time while maintaining acceptable accuracy. Our method completes the search in 10.45 minutes, whereas all baselines take at least $2.7\times$ as long, with Evo-HW-NAS taking $3.79\times$ as long despite achieving the highest accuracy. Moreover, the baselines' runtime excludes the cloud-to-edge communication overhead for transmitting the selected model to the device. Since cloud-only methods also rely entirely on hardware surrogates, the selected architecture may fail to satisfy the actual on-device constraints, potentially triggering additional search rounds and further increasing end-to-end latency. In contrast, our edge-stage refinement uses edge data and direct on-device measurements, enabling fast, deployment-aligned adaptation. While the cloud-only baselines improve accuracy by 2.17% to 5.03%, their substantially higher search cost makes them less suitable for edge settings that require rapid reconfiguration under dynamic operating conditions. In addition, for privacy-sensitive domains such as healthcare (Khalid et al., 2023; Williamson & Prybutok, 2024) and others (Zhang et al., 2025; Sriramalakshmi et al., 2022), public data may be unavailable or substantially misaligned with local data; thus, cloud-only baselines that rely solely on public data for evaluation can select architectures that underperform on-device.

Among the cloud-only baselines in Table 2, Evo-HW-NAS achieves the highest accuracy but also incurs the largest search time, indicating its stronger yet more expensive exploration. RL-HW-NAS offers a balance between search time and utility, achieving 90.2% accuracy in 35.09 min. BO-HW-NAS is the fastest baseline because it uses a Gaussian-process surrogate model to guide architecture selection and typically evaluates fewer architectures than population-based methods. However, it also yields the lowest accuracy likely because the discrete, high-dimensional architecture space makes the Gaussian surrogate and UCB acquisition less accurate, leading to less effective exploration and earlier convergence to suboptimal regions.

Overall, our two-stage design delivers a strong runtime advantage, making re-runnable adaptation practical, while retaining competitive utility. As discussed later in Section 5, this speedup is especially valuable for interactive workloads with concurrent modules and session times on the order of tens of minutes to an hour. Table 2 also highlights a clear tradeoff between search cost and utility across methods.

4.4. Language Modeling

Table 3. LM supernet construction configuration.

Search Space	**Options**
Base network	`GPT-2`
Number of layers	$\{6,\ 9,\ 12\}$
Attention heads number (h)	$\{6,\ 12\}$
FFN ratio ($d_{\text{ffn}}/d_{\text{model}}$)	$\{2.0,\ 3.0,\ 4.0\}$
Dropout ratio	$\{0.0,\ 0.1\}$
Separable Conv kernel sizes	$\{3,\ 5\}$

Table 4. HW-NAS performance on language modeling

Framework	PPL	ΔPPL. (%)	Runtime (min)	ΔTime (%)
Our framework	54.19	-	12.12	-
Evo-HW-NAS	46.21	↓ 14.72	45.37	↑ 274.3
RL-HW-NAS	47.24	↓ 12.83	42.05	↑ 246.9
BO-HW-NAS	50.67	↓ 6.49	36.53	↑ 201.4

Note: We report perplexity (PPL) on Penn Treebank (edge data) as the utility metric for language modeling and runtime as the HW-NAS search time. ΔPPL and ΔTime denote the relative change with respect to our framework.

In language modeling, we build a GPT-style supernet (Radford et al., 2019) and introduce elasticity over depth, attention heads, and FFN expansion ratios, following common Transformer supernet designs (Xu et al., 2021; Chitty-Venkata et al., 2022) (See Table 3). We use perplexity (PPL) as the utility metric and constrain the edgenet with TTFT (prefill) ≤ 300 ms, TPOT (decode) ≤ 10 ms, and memory usage ≤ 1 GB, which are in line with typical latency and memory targets reported for practical language model inference (Agarwal et al., 2023; Arunachalam et al., 2025; openai-community & Hugging Face, 2019). To further bias the search toward efficient architectures, we adopt tighter budgets than these references. The cloud and edge datasets are WikiText-2 and Penn Treebank, respectively, following their standard use as language modeling benchmarks, (Liu et al., 2019a; openai-community & Hugging Face, 2019).

Table 4 exhibits the same trend as the image classification results in Table 2: our framework achieves the lowest HW-NAS search time while maintaining acceptable utility, whereas cloud-only baselines obtain better utility at substantially higher search cost. Specifically, our method completes the search in 12.12 minutes, while all baselines require at least $3\times$ as long, with Evo-HW-NAS taking $3.74\times$ as long while achieving the lowest PPL. As with image classification, baseline runtimes exclude cloud-to-edge communica-

tion overhead, and their reliance on surrogates can create deployment gaps under actual on-device TTFT/TPOT constraints, potentially triggering repeated search cycles and further increasing end-to-end adaptation latency.

As shown in Table 4, the utility gap is larger, and the runtime is longer than in Table 2. This is expected for several reasons. First, PPL is more sensitive than classification accuracy: small degradations in next-token likelihood accumulate over long sequences, producing a larger relative change in PPL even when qualitative generation remains acceptable. Second, evaluating the language model is computationally more intensive than classification due to longer sequences and causal masking, which increase per-architecture evaluation cost and thus total search time. Third, the language model's supernet search space (e.g., depth, heads, FFN ratios, and kernelized variants) is typically more heterogeneous, requiring broader exploration to identify strong architectures and making lightweight edge refinement more challenging. Overall, consistent with Table 2, these results highlight a clear tradeoff between search cost and utility, and our two-stage design prioritizes fast, re-runnable adaptation while retaining competitive language modeling performance.

4.5. Impact of various search configurations

In this section, we perform an ablation study to examine how key search configurations affect the utility–runtime tradeoff of our cloud–edge HW-NAS, focusing on (i) the edge-stage beam width b and (ii) the choice of cloud-stage search algorithm (i.e., evolutionary-based, RL-based, or BO-based search algorithms). Table 5 summarizes the results.

Across both tasks, increasing the edge beam width b consistently improves utility but substantially increases runtime. For image classification, raising b from 1 to 3 improves accuracy by at most 1.4%, while runtime increases from $\sim$ 10 minutes to $\sim$ 35 minutes. Similarly, for language modeling, increasing b from 1 to 3 reduces perplexity by up to 4.76%, while runtime grows from $\sim$ 12 minutes to $\sim$ 37 minutes. In both tasks, the utility exhibit diminishing benefits: most improvement comes from increasing b from 1 to 2 (e.g., 54.19 to 52.27 with Evo-based cloud search in language modeling), whereas the additional gain from $b = 2$ to $b = 3$ is marginal (52.27 to 51.95). Notably, even moving to $b = 2$ nearly doubles the search time, making the extra utility gain relatively modest.

Comparing cloud search algorithms under a fixed beam width b, the runtime is dominated by the edge stage and is similar across cloud search methods. Final utility is also broadly comparable across cloud-stage algorithms after edge refinement. These results suggest that b provides a direct knob for adaptation latency, and that our framework is robust to the choice of cloud-stage search method.

5. Discussion

As shown in Section 4, our framework reduces the end-to-end on-device search time to $\sim$10 minutes by refining architectures on-device. This design alleviates practical deployment issues, such as unstable network connectivity and the overhead of repeated cloud–edge transfers. In contrast, cloud-only HW-NAS baselines typically rely on latency surrogates or measurements collected on idle devices, which often fail to reflect real operating conditions. In practice, on-device inference latency/memory can vary substantially due to co-running workloads (Yang et al., 2020; Wen et al., 2023), and thermal effect (Tan & Cao, 2024). As a result, a cloud-selected architecture may violate latency constraints at deployment time, triggering repeated research and redeployment, which significantly increases the effective runtime overhead beyond what cloud-only search reports in Tables 2 and 4. This runtime reduction is particularly meaningful for interactive workloads that run multiple modules concurrently over sessions lasting tens of minutes to about an hour across repeated interactions (Meta Horizon OS Developers, 2025), such as AR pipelines that co-run SLAM, rendering, and recognition components (Carmigniani & Furht, 2011). Under such conditions, completing adaptation in $\sim$10 minutes is often acceptable, whereas a search lasting over 30 minutes, as shown in Tables 2 and 4, can consume a large fraction of the session and become impractical, especially after accounting for communication and potential repeated search overhead.

In exchange for the reduced search time, our final models achieve 87.5% accuracy and 54.19 perplexity, which is acceptable under strict latency budgets and comparable to efficient models such as MobileNet (Howard et al., 2017) and TinyBERT (Jiao et al., 2020). While our absolute utility is lower than heavily optimized public baselines, those models often benefit from large-scale pretraining, extensive hyperparameter tuning, and distillation. More broadly, HW-NAS (Chitty-Venkata & Somani, 2022; Benmeziane et al., 2021b), which emphasize optimizing architectures for on-device execution rather than unconstrained utility, inherently involves a utility–latency tradeoff.

6. Related Work

Efficient Neural Network. Compact architectures, including the MobileNet family (Howard et al., 2017; Sandler et al., 2019; Howard et al., 2019) and lightweight detectors (Liu et al., 2016), in vision, and compact Transformer variants, such as MobileBERT (Sun et al., 2020) and ALBERT (Lan et al., 2020), in NLP, achieve strong utility–efficiency tradeoffs through manually designed components. However, these hand-crafted models are typically optimized for static deployment settings and do not account for runtime workload variability or shifts between public training data

Table 5. Effect of edge beam width b under different cloud search algorithms.

Edge Beam b \ Cloud Search	Evo-based Search		RL-based Search		BO-based Search	
	Acc. (%)	Runtime (min)	Acc. (%)	Runtime (min)	Acc. (%)	Runtime (min)
$b = 1$	87.5	10.45	86.9	10.61	86.1	10.21
$b = 2$	88.3	21.68	87.6	22.34	87.1	21.56
$b = 3$	88.6	34.79	87.8	34.01	87.3	34.87

(a) Image classification performance under different search configurations.

Edge Beam b \ Cloud Search	Evo-based Search		RL-based Search		BO-based Search	
	PPL	Runtime (min)	PPL	Runtime (min)	PPL	Runtime (min)
$b = 1$	54.19	12.12	55.03	12.41	55.87	12.01
$b = 2$	52.27	25.67	53.78	25.34	54.13	25.61
$b = 3$	51.95	37.90	53.04	37.01	53.21	37.98

(b) Language modeling performance under different search configurations.

and private on-device data. Moreover, manual design, even assisted with NAS (Howard et al., 2019; Tan & Le, 2020), requires significant human effort and relies heavily on prior design experience. In contrast, our approach automatically adapts models to the device's current operating conditions and local data via on-device refinement.

Network Compression Techniques. Model compression methods, such as pruning (Molchanov et al., 2019; Liu et al., 2019b; Frankle & Carbin, 2019; Ashkboos et al., 2024; Ma et al., 2023), knowledge distillation (Hinton et al., 2015; Gou et al., 2021; Gu et al., 2025), and quantization (Gholami et al., 2022; Jacob et al., 2017; Dettmers et al., 2023), have been widely used to improve efficiency. Nonetheless, these techniques typically operate on a fixed backbone and optimize the model for static device conditions using public datasets. Moreover, compression often optimizes proxy metrics (e.g., FLOPs) that do not always translate to real inference time and memory usage on-device, and practical speedups can require specialized hardware support or optimized kernels. In contrast, our mechanism performs architecture refinement using on-device execution measurements during the search process, enabling direct optimization for deployment-time latency and memory while adapting to workload changes and data distribution shifts.

Neural Architecture Search. Neural architecture search (NAS) automates network design, reducing reliance on human intuition while discovering architectures with strong utility. NAS methods span several families, including RL (Jaafra et al., 2019), evolutionary search (Liu et al., 2023), and gradient-based relaxations (Santra et al., 2021). Early approaches (Zoph & Le, 2017; So et al., 2019) demonstrated the potential of NAS but were often prohibitively expensive due to the need to train many candidate models. Later work reduces search cost via weight sharing (e.g., ENAS (Pham et al., 2018) and one/few-shot methods (Su et al., 2021; Wang et al., 2023)) and differentiable optimization (e.g., DARTS (Liu et al., 2019a)). Despite these advances, most NAS pipelines remain best suited to resource-rich, centralized settings. Moreover, NAS primarily optimizes utility without explicitly enforcing hardware constraints such as latency and memory, which motivates hardware-aware NAS.

Hardware-aware Neural Architecture Search. Hardware-aware NAS (HW-NAS) (Chitty-Venkata & Somani, 2022; Benmeziane et al., 2021b;a) explicitly optimizes model utility under deployment constraints such as latency and memory. Similar to general NAS, HW-NAS spans multiple algorithmic families, including RL-based (Tan et al., 2019; Fayyazi et al., 2025), gradient-based (Cai et al., 2019; Sukthanker et al., 2025), and evolutionary-based (Cai et al., 2020; Sinha et al., 2023; Jiang et al., 2021) approaches. They typically incorporate hardware cost into their optimization objective. A key practical challenge in HW-NAS is hardware-cost estimation: many methods rely on surrogate cost models (Wu et al., 2019; Cai et al., 2019; Akhauri & Abdelfattah, 2024; Feng et al., 2024) or proxy metrics (Lu et al., 2019a; Trirat & Lee, 2025). However, these surrogates can deviate from actual behavior due to runtime effects. Moreover, HW-NAS is often conducted in centralized environments using public datasets and fixed benchmarking conditions, which can lead to suboptimal choices on edge devices: on-device data may be distribution-shifted, and real operating conditions can substantially change both latency and peak memory usage. In contrast, our cloud–edge collaborative framework preserves cloud-side scalability while enabling deployment-faithful adaptation: the cloud stage uses only cloud-available data together with an offline hardware surrogate to produce a compact, hardware-feasible candidate subnet that is inexpensive to transmit, and the edge performs lightweight refinement using local data and direct profiling to adapt to current conditions.

7. Conclusion

In this work, we address a practical setting with private on-device data and time-varying device conditions. We propose a cloud–edge HW-NAS framework that trains a supernet and extracts a subnet in the cloud, then efficiently refines it on-device using local data and direct profiling. Experiments show that our method substantially reduces search time while maintaining competitive utility, enabling re-runnable adaptation under time-varying conditions.

References

Acar, A., Aksu, H., Uluagac, A. S., and Conti, M. A survey on homomorphic encryption schemes: Theory and implementation. *ACM Comput. Surv.*, 51(4), July 2018. ISSN 0360-0300. doi: 10.1145/3214303.

Agarwal, M., Qureshi, A., Sardana, N., Li, L., Quevedo, J., and Khudia, D. Llm inference performance engineering: Best practices. https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices, October 2023. URL https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices. Databricks Blog (Mosaic AI Research), Oct. 12, 2023.

Akhauri, Y. and Abdelfattah, M. S. On latency predictors for neural architecture search. In Gibbons, P., Pekhimenko, G., and Sa, C. D. (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 512–523, 2024.

Arunachalam, M., Hodak, M., Palangappa, P., Liao, W.-T., Kannikanti, U., Li, F., Mathews, N., Poornachandran, R., Garvey, E., Deepak, K., Xu, Y., Li, Z., Li, G., Yin, X., Li, D., Lin, Z., Luo, W., Bao, B., Tiwari, S., Zhang, N., Gokhale, V., Greene, C., and Li, E. Technical dive into amd's mlperf inference v5.1 submission, September 2025. URL https://rocm.blogs.amd.com/artificial-intelligence/mlperf-inference-v5.1/README.html. ROCm Blogs. Accessed: 2026-01-22.

Ashkboos, S., Croci, M. L., do Nascimento, M. G., Hoefler, T., and Hensman, J. Slicegpt: Compress large language models by deleting rows and columns, 2024. URL https://arxiv.org/abs/2401.15024.

Benmeziane, H., El Maghraoui, K., Ouarnoughi, H., Niar, S., Wistuba, M., and Wang, N. Hardware-aware neural architecture search: Survey and taxonomy. In *IJCAI*, volume 2021, pp. 4322–4329, 2021a.

Benmeziane, H., Maghraoui, K. E., Ouarnoughi, H., Niar, S., Wistuba, M., and Wang, N. A comprehensive survey on hardware-aware neural architecture search, 2021b. URL https://arxiv.org/abs/2101.09336.

Cai, H., Zhu, L., and Han, S. Proxylessnas: Direct neural architecture search on target task and hardware, 2019. URL https://arxiv.org/abs/1812.00332.

Cai, H., Gan, C., Wang, T., Zhang, Z., and Han, S. Once-for-all: Train one network and specialize it for efficient deployment, 2020. URL https://arxiv.org/abs/1908.09791.

Cai, Z., Chen, L., Liu, P., Ling, T., and Lai, Y. Eg-nas: Neural architecture search with fast evolutionary exploration. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(10):11159–11167, Mar. 2024. doi: 10.1609/aaai.v38i10.28993.

Carmigniani, J. and Furht, B. *Augmented Reality: An Overview*, pp. 3–46. Springer New York, New York, NY, 2011. ISBN 978-1-4614-0064-6. doi: 10.1007/978-1-4614-0064-6_1.

Cheng, Y., Wang, D., Zhou, P., and Zhang, T. A survey of model compression and acceleration for deep neural networks, 2020. URL https://arxiv.org/abs/1710.09282.

Chitty-Venkata, K. T. and Somani, A. K. Neural architecture search survey: A hardware perspective. *ACM Comput. Surv.*, 55(4), November 2022. ISSN 0360-0300. doi: 10.1145/3524500.

Chitty-Venkata, K. T., Emani, M., Vishwanath, V., and Somani, A. K. Neural architecture search for transformers: A survey. *IEEE Access*, 10:108374–108412, 2022. doi: 10.1109/ACCESS.2022.3212767.

Da'u, A. and Salim, N. Recommendation system based on deep learning methods: A systematic review and new directions. *Artificial Intelligence Review*, 53(4):2709–2748, April 2020. doi: 10.1007/s10462-019-09744-1. Published: 03 August 2019.

Deng, L., Li, G., Han, S., Shi, L., and Xie, Y. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proceedings of the IEEE*, 108 (4):485–532, 2020. doi: 10.1109/JPROC.2020.2976475.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. Qlora: Efficient finetuning of quantized llms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 10088–10115. Curran Associates, Inc., 2023.

Fayyazi, A., Kamal, M., and Pedram, M. Marco: Hardware-aware neural architecture search for edge devices with multi-agent reinforcement learning and conformal prediction filtering, 2025. URL https://arxiv.org/abs/2506.13755.

Feng, C., Zhang, L. L., Liu, Y., Xu, J., Zhang, C., Wang, Z., Cao, T., Yang, M., and Tan, H. LitePred: Transferable and scalable latency prediction for Hardware-Aware neural architecture search. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pp. 1463–1477, Santa Clara, CA, April 2024. USENIX Association. ISBN 978-1-939133-39-7.

Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks, 2019. URL https://arxiv.org/abs/1803.03635.

Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., and Keutzer, K. A survey of quantization methods for efficient neural network inference. In *Low-power computer vision*, pp. 291–326. Chapman and Hall/CRC, 2022.

Gou, J., Yu, B., Maybank, S. J., and Tao, D. Knowledge distillation: A survey. *Int. J. Comput. Vis.*, 129(6):1789–1819, June 2021. doi: 10.1007/s11263-021-01453-z.

Gu, Y., Dong, L., Wei, F., and Huang, M. Minillm: Knowledge distillation of large language models, 2025. URL https://arxiv.org/abs/2306.08543.

Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network, 2015. URL https://arxiv.org/abs/1503.02531.

Hong, K., Li, X., Chen, L., Mao, Q., Dai, G., Ning, X., Yan, S., Liang, Y., and Wang, Y. Sola: Optimizing slo attainment for large language model serving with state-aware scheduling. In Zaharia, M., Joshi, G., and Lin, Y. (eds.), *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025.

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., and Adam, H. Searching for mobilenetv3, 2019. URL https://arxiv.org/abs/1905.02244.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017. URL https://arxiv.org/abs/1704.04861.

Jaafra, Y., Luc Laurent, J., Deruyver, A., and Saber Naceur, M. Reinforcement learning for neural architecture search: A review. *Image and Vision Computing*, 89:57–66, 2019. ISSN 0262-8856. doi: https://doi.org/10.1016/j.imavis.2019.06.005.

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference, 2017. URL https://arxiv.org/abs/1712.05877.

Jiang, Q., Zhang, X., Chen, D., Do, M. N., and Yeh, R. A. Eh-dnas: End-to-end hardware-aware differentiable neural architecture search, 2021. URL https://arxiv.org/abs/2111.12299.

Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., Li, L., Wang, F., and Liu, Q. TinyBERT: Distilling BERT for natural language understanding. In Cohn, T., He, Y., and Liu, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4163–4174, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.372.

Khalid, N., Qayyum, A., Bilal, M., Al-Fuqaha, A., and Qadir, J. Privacy-preserving artificial intelligence in healthcare: Techniques and applications. *Computers in Biology and Medicine*, 158:106848, 2023. ISSN 0010-4825. doi: https://doi.org/10.1016/j.compbiomed.2023.106848. URL https://www.sciencedirect.com/science/article/pii/S001048252300313X.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. Albert: A lite bert for self-supervised learning of language representations, 2020. URL https://arxiv.org/abs/1909.11942.

Li, X., Li, Y., Li, Y., Cao, T., and Liu, Y. Flexnn: Efficient and adaptive dnn inference on memory-constrained edge devices. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, ACM MobiCom '24, pp. 709–723, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704895. doi: 10.1145/3636534.3649391.

Liu, H., Simonyan, K., and Yang, Y. Darts: Differentiable architecture search, 2019a. URL https://arxiv.org/abs/1806.09055.

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., and Berg, A. C. *SSD: Single Shot MultiBox Detector*, pp. 21–37. Springer International Publishing, 2016. ISBN 9783319464480. doi: 10.1007/978-3-319-46448-0_2.

Liu, Y., Sun, Y., Xue, B., Zhang, M., Yen, G. G., and Tan, K. C. A survey on evolutionary neural architecture search. *IEEE Transactions on Neural Networks and Learning Systems*, 34(2):550–570, 2023. doi: 10.1109/TNNLS.2021.3100554.

Liu, Z., Sun, M., Zhou, T., Huang, G., and Darrell, T. Rethinking the value of network pruning, 2019b. URL https://arxiv.org/abs/1810.05270.

Lu, Z., Whalen, I., Boddeti, V., Dhebar, Y., Deb, K., Goodman, E., and Banzhaf, W. Nsga-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '19, pp. 419–427, New York, NY, USA, 2019a. Association for Computing Machinery. ISBN 9781450361118. doi: 10.1145/3321707.3321729.

Lu, Z., Whalen, I., Boddeti, V., Dhebar, Y., Deb, K., Goodman, E., and Banzhaf, W. Nsga-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '19, pp. 419–427, New York, NY, USA, 2019b. Association for Computing Machinery. ISBN 9781450361118. doi: 10.1145/3321707.3321729.

Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 21702–21720. Curran Associates, Inc., 2023.

Meta Horizon OS Developers. Building content to optimize user time: 20–40 minute "goldilocks" session length on meta quest. https://developers.meta.com/horizon/blog/optimize-user-time-goldilocks-session-length-meta-quest/, July 2025. Accessed: 2026-01-14.

Molchanov, P., Mallya, A., Tyree, S., Frosio, I., and Kautz, J. Importance estimation for neural network pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

openai-community and Hugging Face. openai-community/gpt2. https://huggingface.co/openai-community/gpt2, 2019. Hugging Face model card. Accessed: 2026-01-22.

Pham, H., Guan, M., Zoph, B., Le, Q., and Dean, J. Efficient neural architecture search via parameters sharing. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 4095–4104. PMLR, 10–15 Jul 2018.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Russell, S. J. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, USA, 4 edition, 2021. ISBN 9780134610993.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks, 2019. URL https://arxiv.org/abs/1801.04381.

Santra, S., Hsieh, J.-W., and Lin, C.-F. Gradient descent effects on differential neural architecture search: A survey. *IEEE Access*, 9:89602–89618, 2021. doi: 10.1109/ACCESS.2021.3090918.

Sinha, N., Shabayek, A. E. R., Kacem, A., Rostami, P., Shneider, C., and Aouada, D. Hardware aware evolutionary neural architecture search using representation similarity metric, 2023. URL https://arxiv.org/abs/2311.03923.

So, D., Le, Q., and Liang, C. The evolved transformer. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5877–5886. PMLR, 09–15 Jun 2019.

Sriramalakshmi, P., Rajkumar, S. P., and Nivedhithaa, R. Modern machine learning and iot applications for personalized healthcare: Opportunities and challenges. *Transformation in Healthcare with Emerging Technologies*, pp. 199–216, 2022.

Su, X., You, S., Zheng, M., Wang, F., Qian, C., Zhang, C., and Xu, C. K-shot nas: Learnable weight-sharing for nas with k-shot supernets. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9880–9890. PMLR, 18–24 Jul 2021.

Sukthanker, R. S., Zela, A., Staffler, B., Dooley, S., Grabocka, J., and Hutter, F. Multi-objective differentiable neural architecture search, 2025. URL https://arxiv.org/abs/2402.18213.

Sun, Z., Yu, H., Song, X., Liu, R., Yang, Y., and Zhou, D. Mobilebert: a compact task-agnostic bert for resource-limited devices, 2020. URL https://arxiv.org/abs/2004.02984.

Tan, M. and Le, Q. V. Efficientnet: Rethinking model scaling for convolutional neural networks, 2020. URL https://arxiv.org/abs/1905.11946.

Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., and Le, Q. V. Mnasnet: Platform-aware neural architecture search for mobile. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

Tan, T. and Cao, G. Thermal-aware scheduling for deep learning on mobile devices with npu. *IEEE Transactions on Mobile Computing*, 23(12):10706–10719, 2024. doi: 10.1109/TMC.2024.3379501.

Trirat, P. and Lee, J.-G. Monaq: Multi-objective neural architecture querying for time-series analysis on resource-constrained devices, 2025. URL https://arxiv.org/abs/2505.10607.

Wang, H., Ge, C., Chen, H., and Sun, X. PreNAS: Preferred one-shot learning towards efficient neural architecture search. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 35642–35654. PMLR, 23–29 Jul 2023.

Wen, H., Li, Y., Zhang, Z., Jiang, S., Ye, X., Ouyang, Y., Zhang, Y., and Liu, Y. Adaptivenet: Post-deployment neural architecture adaptation for diverse edge environments. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, ACM MobiCom '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450399906. doi: 10.1145/3570361.3592529.

Williamson, S. M. and Prybutok, V. Balancing privacy and progress: A review of privacy challenges, systemic oversight, and patient perceptions in ai-driven healthcare. *Applied Sciences*, 14(2), 2024. ISSN 2076-3417. doi: 10.3390/app14020675. URL https://www.mdpi.com/2076-3417/14/2/675.

Wu, B., Dai, X., Zhang, P., Wang, Y., Sun, F., Wu, Y., Tian, Y., Vajda, P., Jia, Y., and Keutzer, K. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

Xu, J., Tan, X., Luo, R., Song, K., Li, J., Qin, T., and Liu, T.-Y. Nas-bert: Task-agnostic and adaptive-size bert compression with neural architecture search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD '21, pp. 1933–1943, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383325. doi: 10.1145/3447548.3467262.

Xue, Y., Zha, J., Pelusi, D., Chen, P., Luo, T., Zhen, L., Wang, Y., and Wahib, M. Neural architecture search with progressive evaluation and subpopulation preservation. *IEEE Transactions on Evolutionary Computation*, 29(5): 1678–1691, 2025. doi: 10.1109/TEVC.2024.3393304.

Yang, L., Lu, B., and Ren, S. A note on latency variability of deep neural networks for mobile inference, 2020. URL https://arxiv.org/abs/2003.00138.

Yang, T.-J., Howard, A., Chen, B., Zhang, X., Go, A., Sandler, M., Sze, V., and Adam, H. Netadapt: Platform-aware neural network adaptation for mobile applications. In *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.

Yin, L., Liu, S., Fang, M., Huang, T., Menkovski, V., and Pechenizkiy, M. Lottery pools: Winning more by interpolating tickets without increasing training or inference cost. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9):10945–10953, Jun. 2023. doi: 10.1609/aaai.v37i9.26297.

Ying, C., Klein, A., Christiansen, E., Real, E., Murphy, K., and Hutter, F. NAS-bench-101: Towards reproducible neural architecture search. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 7105–7114. PMLR, 09–15 Jun 2019.

Yu, J. and Huang, T. S. Universally slimmable networks and improved training techniques. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

Yuan, G., Xue, B., and Zhang, M. An evolutionary neural architecture search method based on performance prediction and weight inheritance. *Information Sciences*, 667:120466, 2024. ISSN 0020-0255. doi: https://doi.org/10.1016/j.ins.2024.120466.

Zhang, J., Segueria, R., and Psounis, K. Spinml: Customized synthetic data generation for private training of specialized ml models. *Proceedings on Privacy Enhancing Technologies*, 2025.

Zhao, Z.-Q., Zheng, P., Xu, S.-T., and Wu, X. Object detection with deep learning: A review. *IEEE Transactions on Neural Networks and Learning Systems*, 30(11):3212–3232, 2019. doi: 10.1109/TNNLS.2018.2876865.

Zoph, B. and Le, Q. V. Neural architecture search with reinforcement learning, 2017. URL https://arxiv.org/abs/1611.01578.